DSA Experiment 4

Title: Implement Database Management using arrays of structures with operations Create, Display, Modify,Search and Sort.

Name- Mokshada Hejib **PRN**-B25ET1013 **Div**-SY1 A

```
#include <stdio.h>
#include <string.h>

typedef struct student
{
   int roll_no;
   char name[100];
   char class;
   int marks;
} student;

void insert(student S[], int *n, int p);
void delete(student S[], int *n, int p);
int search(student S[], int r, int n);
void display(student S[], int n);
void create(student S[], int n);
void sort(student S[], int n);
void modify(student S[], int n);

int main()
{
   student S[100];
   int choice, p, r, n = 0, position;

   do
   {
      printf("\nSTUDENT RECORD SYSTEM\n");
      printf("1. Create\n");
      printf("2. Insert\n");
      printf("3. Search\n");
      printf("4. Sort\n");
      printf("5. Modify\n");
      printf("6. Delete\n");
      printf("7. Display\n");
```

```
printf("8. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
   case 1:
      printf("Enter number of students: ");
      scanf("%d", &n);

      create(S, n);
      break;

   case 2:
      printf("Enter position: ");
      scanf("%d", &p);

      insert(S, &n, p);
      break;

   case 3:
      printf("Enter roll number to search: ");
      scanf("%d", &r);

      position = search(S, r, n);

      if(position != -1)
      {
         printf("Student found at position %d\n", position + 1);
         printf("Roll No: %d\n", S[position].roll_no);
         printf("Name: %s\n", S[position].name);
         printf("Class: %c\n", S[position].class);
         printf("Marks: %d\n", S[position].marks);
      }
      else
      {
         printf("Student not found.\n");
      }
      break;

   case 4:
      sort(S, n);
      printf("Students sorted successfully.\n");
```

```
            break;

        case 5:
            modify(S, n);
            break;

        case 6:
            printf("Enter position to delete: ");
            scanf("%d", &p);

            delete(S, &n, p);
            break;

        case 7:
            display(S, n);
            break;

        case 8:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice.\n");
      }

   } while(choice != 8);

   return 0;
}


void create(student S[], int n)
{
   int i;

   for(i = 0; i < n; i++)
   {
      printf("\nEnter details of student %d:\n", i + 1);

      printf("Enter name: ");
      scanf("%s", S[i].name);

      printf("Enter roll number: ");
      scanf("%d", &S[i].roll_no);
```

```
        printf("Enter class: ");
        scanf(" %c", &S[i].class);

        printf("Enter marks: ");
        scanf("%d", &S[i].marks);
    }
}


void insert(student S[], int *n, int p)
{
    int i;

    if(*n >= 100)
    {
        printf("Student list is full.\n");
        return;
    }

    if(p < 1 || p > *n + 1)
    {
        printf("Invalid position.\n");
        return;
    }

    for(i = *n; i >= p; i--)
    {
        S[i] = S[i - 1];
    }

    printf("\nEnter details of new student:\n");

    printf("Enter name: ");
    scanf("%s", S[p - 1].name);

    printf("Enter roll number: ");
    scanf("%d", &S[p - 1].roll_no);

    printf("Enter class: ");
    scanf(" %c", &S[p - 1].class);

    printf("Enter marks: ");
    scanf("%d", &S[p - 1].marks);
```

```
    (*n)++;

    printf("Student inserted successfully.\n");
}


int search(student S[], int r, int n)
{
    int i;

    for(i = 0; i < n; i++)
    {
        if(S[i].roll_no == r)
        {
            return i;
        }
    }

    return -1;
}


void sort(student S[], int n)
{
    int i, j;
    student temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(S[j].roll_no > S[j + 1].roll_no)
            {
                temp = S[j];
                S[j] = S[j + 1];
                S[j + 1] = temp;
            }
        }
    }
}


void modify(student S[], int n)
```

```
{
    int r, position;

    printf("Enter roll number of student to modify: ");
    scanf("%d", &r);

    position = search(S, r, n);

    if(position == -1)
    {
        printf("Student not found.\n");
        return;
    }

    printf("\nEnter new details:\n");

    printf("Enter name: ");
    scanf("%s", S[position].name);

    printf("Enter roll number: ");
    scanf("%d", &S[position].roll_no);

    printf("Enter class: ");
    scanf(" %c", &S[position].class);

    printf("Enter marks: ");
    scanf("%d", &S[position].marks);

    printf("Student modified successfully.\n");
}


void delete(student S[], int *n, int p)
{
    int i;

    if(p < 1 || p > *n)
    {
        printf("Invalid position.\n");
        return;
    }

    for(i = p - 1; i < *n - 1; i++)
    {
```

```
        S[i] = S[i + 1];
    }

    (*n)--;

    printf("Student deleted successfully.\n");
}


void display(student S[], int n)
{
    int i;

    if(n == 0)
    {
        printf("No student records available.\n");
        return;
    }

    printf("\nSTUDENT RECORDS-\n");

    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("Roll No : %d\n", S[i].roll_no);
        printf("Name    : %s\n", S[i].name);
        printf("Class   : %c\n", S[i].class);
        printf("Marks   : %d\n", S[i].marks);
    }
}
```

OUTPUT:

Output

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter number of students: 2

Enter details of student 1:
Enter name: Mokshada
Enter roll number: 13
Enter class: A
Enter marks: 88

Enter details of student 2:
Enter name: Mugdha
Enter roll number: 22
Enter class: A
Enter marks: 92
```

Output

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 2
Enter position: 2

Enter details of new student:
Enter name: Mansi
Enter roll number: 33
Enter class: A
Enter marks: 94
Student inserted successfully.
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 3
Enter roll number to search: 13
Student found at position 1
Roll No: 13
Name: Mokshada
Class: A
Marks: 88
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 4
Students sorted successfully.
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 5
Enter roll number of student to modify: 22

Enter new details:
Enter name: Mugdha
Enter roll number: 22
Enter class: A
Enter marks: 95
Student modified successfully.
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 6
Enter position to delete: 3
Student deleted successfully.
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 7

STUDENT RECORDS-

Student 1
Roll No : 13
Name    : Mokshada
Class   : A
Marks   : 88

Student 2
Roll No : 33
Name    : Mansi
Class   : A
Marks   : 94
```

```
STUDENT RECORD SYSTEM
1. Create
2. Insert
3. Search
4. Sort
5. Modify
6. Delete
7. Display
8. Exit
Enter your choice: 8
Exiting program...
```